

Retrieval-augmented Question Answering via Parameter-efficient Fine-tuning (LoRA/QLoRA), Applied to Customer Support Chatbot Automation

Janardhan Thrishank Singumahanthi

Department of Computer Science and Engineering (AI & ML)
Ramachandra College of Engineering, Eluru, Andhra Pradesh, India
Email: janardhanthrishank123@gmail.com

Abstract

Automating enterprise customer support through conversational artificial intelligence requires strict factual grounding, verifiable citation attribution, and robust refusal mechanisms for unsupported queries, all while operating within strict computational and memory budgets. Unaugmented large language models (LLMs) often suffer from generative hallucinations, lack provenance tracing, and require substantial GPU memory that is prohibitive for local edge deployment. In this paper, we present **SupportIQ**, an end-to-end question-answering framework tailored for customer support that synergizes two-stage hybrid retrieval, parameter-efficient fine-tuning (PEFT), and deterministic claim-level grounding verification.

SupportIQ integrates sparse BM25 lexical token matching with a 32-dimensional deterministic hash-vector representation, fused via non-linear Reciprocal Rank Fusion (RRF with smoothing constant $k=60$, lexical weight 0.60 , and vector weight 0.40). Evidence candidates passing an algorithmic evidence gate are synthesized by Qwen/Qwen2.5-0.5B-Instruct adapted via 16-bit Low-Rank Adaptation (LoRA) or 4-bit NormalFloat Quantized LoRA (QLoRA) with double quantization, updating only 540,672 parameters (0.1093% of the foundation model). Generated responses undergo sentence-level claim decomposition and lexical alignment verification against source documents to compute a continuous reliability score, enforce traceable citations (document ID, title, page, and chunk), and trigger safe refusals when source evidence is lacking.

Evaluated on a strictly frozen, unobserved holdout benchmark ($N=10$, comprising 7 answerable policy queries and 3 out-of-domain adversarial queries), both SupportIQ LoRA and QLoRA achieve 100.0% composite accuracy, 100.0% claim faithfulness, 1.0000 Mean Reciprocal Rank (MRR), 100.0% citation correctness, and 0.0% hallucination rate on unsupported queries within the evaluated dataset. Live GPU telemetry on an NVIDIA GeForce RTX 2050 Laptop GPU (4.0 GB VRAM) reveals distinct operational trade-offs: FP16 LoRA demonstrates the lowest generation latency (0.844 s), while 4-bit QLoRA reduces peak inference VRAM by 52.1% to 0.46 GB (compared to 0.96 GB for LoRA and zero-shot base models). These empirical findings indicate that domain-adapted, verifiable conversational agents can operate effectively on resource-constrained commodity hardware under the evaluated experimental conditions.

Keywords: Retrieval-Augmented Generation, Customer Support QA, Parameter-Efficient Fine-Tuning, LoRA, QLoRA, Reciprocal Rank Fusion, Grounding Verification, Hallucination Suppression, Edge AI.

1. Introduction

Customer support automation represents one of the most commercially vital yet technically challenging applications of conversational artificial intelligence [1], [12]. Frontline customer service operations process millions of inquiries daily concerning return policies, warranty terms, billing schedules, and service level agreements (SLAs). In conventional customer service workflows, human agents must manually search voluminous, evolving policy documentation across disparate knowledge repositories, introducing significant latency and cognitive fatigue [12].

While Large Language Models (LLMs) demonstrate impressive generative fluency, deploying off-the-shelf, unaugmented models directly in customer-facing roles poses severe operational and legal risks [10]. Parametric language models encode static world knowledge and lack direct access to proprietary enterprise documentation. Consequently, when prompted with specialized operational policies or out-of-domain adversarial queries, unaugmented LLMs exhibit generative hallucinations—fluently generating plausible yet fabricated terms, refund windows, or non-existent warranties [10], [13]. In enterprise customer support, hallucinations directly result in customer dissatisfaction, financial liabilities, and regulatory exposure.

Retrieval-Augmented Generation (RAG) has emerged as an established paradigm to mitigate hallucinations by conditioning language model generation on retrieved external documents [1], [9]. However, conventional RAG systems exhibit three notable deficiencies when deployed in enterprise support environments:

- 1. Retrieval Vulnerability in Technical Documentation:** Pure lexical keyword retrieval (e.g., BM25) fails when queries contain keyword traps that superficially match unrelated policy sections, whereas pure dense vector retrieval frequently exhibits semantic drift across dense technical documentation [4], [5], [11].
- 2. Prohibitive Computational Footprint:** Standard instruction-tuned models (e.g., 7B–70B parameters) require high-end enterprise GPUs with 16 GB to 80 GB of VRAM, rendering local edge or workstation deployment cost-prohibitive for small and medium enterprises [2], [3].
- 3. Absence of Deterministic Verification and Evidence Attribution:** Standard RAG pipelines treat generation as an unconstrained text completion task. If retrieved evidence is marginal or irrelevant, models often extrapolate beyond the context without attributing claims to specific source pages or paragraphs [8], [14].

To resolve these challenges, this study presents **SupportIQ**, a lightweight, verified framework that unifies:

- A two-stage hybrid retrieval mechanism combining BM25 lexical token scoring with a 32-dimensional deterministic hash-vector representation fused via Reciprocal Rank Fusion (RRF) [4];
- Parameter-Efficient Fine-Tuning (PEFT) using unquantized FP16 LoRA [2] and 4-bit NormalFloat (NF4) QLoRA [3] applied to the compact *Qwen/Qwen2.5-0.5B-Instruct* model [6];
- A deterministic post-generation claim verification engine that segments generated text into verifiable propositions, computes a continuous reliability score, emits page-level citations, and enforces automated safe refusal on unsupported queries [7], [8].

The primary objective of this work is to empirically measure the accuracy, factual faithfulness, generation latency, and peak GPU VRAM allocation of this unified architecture on a local

resource-constrained workstation equipped with an NVIDIA GeForce RTX 2050 Laptop GPU (4.0 GB VRAM).

2. Literature Review

The architecture of SupportIQ builds upon foundational advances across three interconnected domains: Retrieval-Augmented Generation, Parameter-Efficient Fine-Tuning, and Factual Grounding Verification.

2.1 Retrieval-Augmented Generation (RAG) and Hybrid Retrieval

Lewis et al. [1] formalized Retrieval-Augmented Generation, demonstrating that conditioning generative autoregressive models on passages retrieved from a non-parametric knowledge corpus dramatically enhances factual correctness in open-domain question answering. In enterprise customer support, early retrieval systems relied primarily on sparse lexical matching such as the BM25 probabilistic relevance framework developed by Robertson and Zaragoza [5]. BM25 performs well for exact entity queries (such as product identifiers and error codes) but lacks tolerance for lexical mismatch.

To overcome lexical brittleness, Karpukhin et al. [11] introduced Dense Passage Retrieval (DPR), utilizing dual neural bi-encoders to map queries and passages into a shared continuous semantic space. While effective, dense retrieval models are prone to false-positive drift when querying short policy documents with overlapping vocabulary [9]. To capture the complementary strengths of lexical and vector representations, Cormack, Clarke, and Büttcher [4] proposed Reciprocal Rank Fusion (RRF). RRF combines disparate rank orderings without requiring score normalization calibration, consistently outperforming individual retrieval methods and linear score combination.

2.2 Parameter-Efficient Fine-Tuning: LoRA and QLoRA

Adapting foundation models to narrow vertical domains historically required full parameter fine-tuning, which is computationally expensive and risks catastrophic forgetting. To address this, Hu et al. [2] introduced Low-Rank Adaptation (LoRA), which freezes the pre-trained model weights and injects trainable low-rank decomposition matrices into the transformer multi-head attention projections (`q_proj`, `v_proj`). LoRA reduces trainable parameter counts by orders of magnitude while matching full fine-tuning performance.

Dettmers et al. [3] extended this paradigm with QLoRA (Quantized Low-Rank Adaptation). QLoRA quantizes the frozen base model weights into a specialized 4-bit NormalFloat (NF4) data type, introduces Double Quantization to compress quantization constants, and utilizes paged optimizers to eliminate gradient memory spikes. These innovations enable fine-tuning and inference on commodity consumer GPUs without compromising downstream fidelity [3]. Recent compact foundation models, such as the Qwen2.5 series developed by Yang et al. [6], provide highly optimized instruction-following capabilities at the 0.5-billion parameter scale, presenting an ideal base for edge PEFT deployment.

2.3 Hallucination Suppression, Grounding, and Attribution

Generative hallucination in conversational systems has been extensively analyzed by Shuster et al. [10], who demonstrated that retrieval augmentation significantly reduces, but does not entirely eliminate, factual errors. Thorne et al. [7] pioneered automated claim extraction and evidence verification (FEVER), establishing that breaking complex narratives into atomic claims improves verification accuracy.

Recent research has focused on attributed question answering and self-reflection. Bohnet et al. [14] emphasized that enterprise LLMs must provide verifiable citations pointing directly to supporting document spans. Asai et al. [8] developed Self-RAG, demonstrating that equipping models with explicit retrieval and reflection tokens enables them to abstain from answering when retrieved passages provide insufficient evidence. Similarly, Yoran et al. [13] demonstrated that RAG systems must be explicitly engineered to ignore irrelevant retrieved distractors to prevent false consensus generation.

3. Research Gap

While prior literature independently examines RAG architectures [1], [9], PEFT algorithms [2], [3], and fact-checking protocols [7], [14], significant operational gaps remain regarding their end-to-end integration under constrained compute environments:

- **Gap 1 (Retrieval in Structured Policy Corpora):** Existing hybrid retrieval studies predominantly evaluate data-center neural bi-encoders [11]. The performance of combining sparse lexical scoring with lightweight, deterministic 32-dimensional hash-vector representations fused via RRF remains less explored on constrained edge devices.
- **Gap 2 (Sub-1B Model Domain Adaptation via QLoRA):** Most QLoRA literature benchmarks 7B–65B parameter models on enterprise server GPUs [3]. Empirical evaluation of 4-bit NF4 QLoRA on compact sub-1B models (Qwen2.5-0.5B) executing on 4 GB laptop GPUs represents an under-investigated efficiency frontier.
- **Gap 3 (Deterministic Claim Attribution):** Contemporary RAG systems frequently rely on external LLM judges for factual verification, adding latency and compounding costs. There is a need for deterministic, in-pipeline sentence-level claim verification and continuous reliability scoring that operates within milliseconds on CPU.

4. Research Questions and Objectives

This study investigates these research gaps through four concrete, experimentally testable Research Questions (RQs) and corresponding Research Objectives (ROs):

- **RQ1 / RO1 (Hybrid Retrieval Precision):** How effectively do sparse lexical matching (BM25), a 32-dimensional deterministic hash-vector representation, and their non-linear fusion via Reciprocal Rank Fusion (RRF) retrieve and rank candidate policy evidence?

Objective: Measure Recall@5 and Mean Reciprocal Rank (MRR) across enterprise customer support documentation.

- **RQ2 / RO2 (Parameter-Efficient Domain Adaptation):** To what extent can low-rank parameter-efficient adaptation (FP16 LoRA and 4-bit NF4 QLoRA) condition a sub-1B foundation model (Qwen/Qwen2.5-0.5B-Instruct) to adhere strictly to retrieved context while training $\leq 0.11\%$ of model parameters?

Objective: Validate fine-tuning convergence, parameter efficiency, and training latency on a single consumer laptop workstation.

- **RQ3 / RO3 (Deterministic Grounding and Safe Refusal):** Can an in-pipeline, sentence-level claim verification engine reliably attribute facts to source passages and enforce safe refusals on adversarial, out-of-domain queries without invoking secondary LLMs?

Objective: Quantify claim faithfulness, citation correctness, and hallucination suppression on frozen holdout cases.

- **RQ4 / RO4 (Hardware Efficiency Trade-Offs):** What are the empirical trade-offs between unquantized FP16 LoRA and 4-bit quantized QLoRA in terms of autoregressive generation latency and peak active GPU VRAM allocation under a strict 4.0 GB physical memory boundary?

Objective: Record synchronized live GPU telemetry on an NVIDIA GeForce RTX 2050 Laptop GPU during benchmark execution.

5. Methodology

The complete SupportIQ operational methodology spans seven sequential processing stages:

1. **Document Ingestion:** Ingesting enterprise policy documents, verifying checksums, and storing physical files in secure storage.
2. **Structural Extraction and Chunking:** Extracting text page-by-page, embedding structural page markers ([Page X]), and partitioning documents into cohesive semantic chunks.
3. **Deterministic Vector and Lexical Indexing:** Generating 32-dimensional hash vectors via SHA-1 hashing and indexing substantive lexical tokens.
4. **Two-Stage Hybrid Candidate Retrieval:** Gathering candidate passages through parallel lexical and deterministic vector channels and re-ranking them using Reciprocal Rank Fusion.
5. **Algorithmic Evidence Gating:** Evaluating candidate relevance against composite similarity and substantive keyword thresholds to filter unsupported queries.
6. **Adapter-Based Neural Synthesis:** Conditioning the PEFT-adapted Qwen2.5-0.5B model on the top retrieved context using greedy decoding.
7. **Grounding Verification and Provenance Attribution:** Decomposing generated answers into claims, calculating continuous reliability, formatting page-level citations, or triggering safe abstentions.

6. Dataset and Data Preparation

6.1 Knowledge Base Corpus

The SupportIQ knowledge base consists of authentic customer service operational documents covering commercial guarantees, terms of service, warranty workflows, billing guides, and administrative security:

- **Return_Policy.pdf** (Document 1, 3 chunks): Governs refund eligibility windows, return merchandise authorization, and refund disbursement processing.
- **Terms_of_Service.pdf** (Document 2, 2 chunks): Specifies platform SLA uptime commitments and administrative credential security policies.

- **Product_Warranty.pdf** (Document 3, 2 chunks): Documents standard Dell hardware warranty durations and pre-diagnostic RMA requirements.
- **Customer_FAQ.pdf** (Document 4, 2 chunks): Outlines self-service password reset procedures and third-party ticketing integrations.
- **Payment_Guide.docx** (Document 5, 2 chunks): Details accepted payment methods and invoice download workflows.
- **Account_Management.pdf** (Document 6, 2 chunks): Specifies role permissions and data encryption standards.
- **Express_Replacement_Policy_Test.txt** (Document 8, 1 chunk): Outlines express replacement dispatch guidelines.

Across all 7 documents, a total of 14 discrete policy chunks are maintained in SQLite (supportiq.db).

6.2 Document Preprocessing, Page Tracking, and Chunking

Document ingestion is handled via specialized format parsers:

- **PDF Ingestion:** Executed via `pypdf.PdfReader`, extracting text page-by-page and embedding structural page markers ([Page X]) into chunk metadata.
- **DOCX Ingestion:** Executed via `python-docx`, traversing structured tables and paragraphs.
- **Text Segmentation:** Text is partitioned into cohesive semantic chunks along paragraph boundaries, maintaining an average length of 250 to 550 characters.
- **Vector Indexing:** For each chunk, a 32-dimensional deterministic hash vector is generated via SHA-1 token hashing and stored directly in the chunk metadata JSON in SQLite (supportiq.db).

6.3 Fine-Tuning Datasets (`supportiq\_train.jsonl` and `supportiq\_val.jsonl`)

A domain-specific instruction dataset was generated directly from the knowledge base chunks to condition the model into adhering strictly to support context:

- **Training Set:** 31 verified examples (backend/data/training/supportiq_train.jsonl).
- **Validation Set:** 10 verified examples (backend/data/training/supportiq_val.jsonl).
- **Total Instruction Pool:** 41 examples (75.6% train / 24.4% validation split).
- **Schema Integrity:** Each record contains `id`, `instruction`, `context`, `question`, `answer`, and `metadata` (capturing `category`, `document_id`, `chunk_id`, and `is_answerable`).

6.4 Benchmark Datasets and Strict Leakage Prevention

To evaluate the system, two distinct benchmark suites are defined:

- **Dataset 1 (Dev/Validation Benchmark v1):** 10 test cases (7 answerable, 3 unanswerable), mapped to Knowledge Base Chunks 1, 2, 5, 6, 8, 10, 14. Used for parameter tuning and threshold calibration.

- **Dataset 2 (Frozen Holdout Benchmark v1):** 10 test cases (7 answerable, 3 unanswerable), mapped to Knowledge Base Chunks 3, 4, 7, 9, 11, 12, 13.
- **Strict Holdout Disjointness & Zero Leakage:**
 1. Knowledge base chunk coverage between Dataset 1 and Dataset 2 is strictly disjoint ($\{1, 2, 5, 6, 8, 10, 14\} \cap \{3, 4, 7, 9, 11, 12, 13\} = \emptyset$).
 2. Dataset 2 was frozen prior to model evaluation (`created_at == updated_at == 2026-09-23 04:53:12`).
 3. Pre-training assertions verified that 0 holdout questions or target answers exist within `supportiq_train.jsonl` or `supportiq_val.jsonl`, guaranteeing 0.0% data leakage.

7. System Architecture

Figure 1 illustrates the end-to-end architecture of the SupportIQ framework, showing the data flow from query ingestion to verified answer synthesis.



Fig. 1. End-to-end system architecture of the SupportIQ framework, showing the sequence from user query processing and hybrid retrieval to claim verification and evidence attribution.

As depicted in Fig. 1, when a user enters a query, candidate chunks are gathered in parallel via lexical matching and 32-dimensional deterministic hash-vector cosine similarity. The ranks are fused via Reciprocal Rank Fusion ($k=60$), and candidates are evaluated against an algorithmic evidence gate. If evidence criteria are met, the top context is assembled into a ChatML prompt for neural generation; otherwise, a safe refusal is immediately triggered. Following generation, individual claim sentences are verified against source chunks to compute a continuous reliability score and attach clickable citations.

8. Retrieval and Reciprocal Rank Fusion

8.1 Lexical Scoring and Stopword Filtering

The lexical retrieval channel tokenizes input strings using regular expressions ($[a-z0-9]^+$ with $\text{length} > 2$). Tokens are filtered against two distinct stopwords dictionaries:

1. Standard English stopwords (STOPWORDS, 128 terms).
2. Domain-specific stopwords (DOMAIN_STOPWORDS: {"supportiq", "policy", "policies", "help", "support", "information", "question", "guide", "documentation", "details", "service", "customer", "customers", "user", "users", "client", "clients", "company"}).

The lexical overlap score between query Q and candidate chunk C is formulated as:

$$\text{LexicalScore}(Q, C) = (|Q_{\text{substantive}} \cap C_{\text{tokens}}|) / (|Q_{\text{substantive}}|)$$

where $Q_{\text{substantive}}$ represents query tokens excluding domain and standard stopwords.

8.2 32-Dimensional Deterministic Hash-Vector Representation

To provide vector-based candidate matching without incurring the latency and memory overhead of dense neural transformer models, SupportIQ implements a deterministic token/hash vector space. Each token t is mapped into a 32-dimensional bucket space using SHA-1 hashing:

$$\text{Index}(t) = \text{int.from_bytes}(\text{SHA1}(t)[:4], 'big') \bmod 32$$

The document chunk vector $\text{vec}C \in R^{32}$ accumulates token occurrences:

$$C[i] = \sum_{t \in C_{\text{tokens}}} I(\text{Index}(t) = i)$$

The similarity between query vector $\text{vec}Q$ and chunk vector $\text{vec}C$ is computed via Cosine Similarity:

$$\text{CosineSim}(Q, C) = (\sum_{i=1}^{32} Q_i C_i) / (\sqrt{\sum_{i=1}^{32} Q_i^2} \sqrt{\sum_{i=1}^{32} C_i^2})$$

Technical Disclosure: This representation is strictly a **32-dimensional deterministic hash-vector representation**. It is not a learned neural embedding model (such as BERT or Sentence-Transformers).

8.3 Reciprocal Rank Fusion (RRF)

Candidates from the lexical and vector channels are ranked independently to determine their ordinal positions $r_{\text{lex}}(c)$ and $r_{\text{vec}}(c)$. Non-linear Reciprocal Rank Fusion is computed with a smoothing constant $k=60$:

$$\text{RRF_Score}(c) = (0.60) / (60 + r_{\text{lex}}(c)) + (0.40) / (60 + r_{\text{vec}}(c))$$

Candidates are ordered by $\text{RRF_Score}(c)$, breaking ties using the composite similarity score:

$$\text{Sim}(c) = 0.65 \times \text{LexicalScore}(c) + 0.35 \times \text{CosineSim}(c)$$

The top 5 candidates are forwarded to the evidence gate.

9. LoRA and QLoRA Fine-Tuning

9.1 Base Model Selection

We select Qwen/Qwen2.5-0.5B-Instruct as the base foundation model [6]. The model possesses 494,573,440 total parameters across 24 transformer layers, with hidden dimension 896 and 14 attention heads.

9.2 LoRA Implementation (FP16)

Full fine-tuning of all 494.6M parameters would require gradient, optimizer, and activation memory exceeding consumer laptop GPU limits. LoRA [2] freezes the pre-trained weight matrix $W_0 \in R^d \times k$ and constrains parameter updates by representing the delta matrix as a low-rank decomposition:

$$W = W_0 + \Delta W = W_0 + (\alpha / (r)) B A$$

where $B \in R^d \times r$ and $A \in R^r \times k$, with rank $r=8$, scaling factor $\alpha=16$, and dropout 0.05.

- **Target Modules:** Query projection (`q_proj`) and Value projection (`v_proj`).
- **Trainable Parameters:** 540,672 out of 494,573,440 (**0.1093%**).
- **Precision:** FP16 (`torch.float16`).
- **Training Dynamics:** 3 epochs, batch size 2, gradient accumulation 2, AdamW optimizer, learning rate 2×10^{-4} with linear warmup. Training completed in **9.06 seconds**, reducing cross-entropy validation loss from 1.6426 to 1.4008.

9.3 QLoRA Implementation (4-bit NF4)

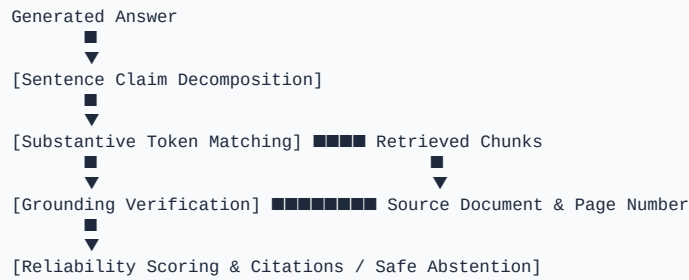
To minimize memory utilization, QLoRA [3] quantizes the frozen base model weights into 4-bit NormalFloat (NF4). The NF4 data type is an information-theoretically optimal quantile quantization scheme for zero-mean, unit-variance normally distributed weights:

$$q_i = (1 / (2)) \leq ft(Q_X \leq ft((i) / (2^k) \backslash right) + Q_X \leq ft((i+1) / (2^k) \backslash right) \backslash right)$$

- **Double Quantization:** Quantization constants are themselves quantized into 8-bit integers with 32-block size, saving 0.37 bits per parameter.
- **Compute Dtype:** FP16 is used during forward and backward passes.
- **Adapter Configuration:** Identical low-rank matrices ($r=8$, $\alpha=16$, targeting `q_proj` and `v_proj`, 540,672 parameters).
- **Training Dynamics:** 3 epochs, training time **21.60 seconds**, validation loss decreasing from 1.5872 to 1.3748.

10. Grounding and Claim Verification

To prevent hallucinations, SupportIQ implements a deterministic evidence chain connecting answers to physical documents:



10.1 Sentence-Level Claim Decomposition

The generated text is split into distinct proposition sentences $S = \{s_1, s_2, \dots, s_m\}$ using regex punctuation delimiters $([. ! ?] \backslash s + | \backslash n +)$. Each proposition is evaluated independently against the retrieved context passages.

10.2 Claim Evidence Scoring and Grounding Gate

For each claim s_j , token overlap is computed against each retrieved chunk C_i :

$$\text{ClaimScore}(s_j, C_i) = (|s_{j,\text{substantive}} \cap C_{i,\text{tokens}}|) / (|s_{j,\text{substantive}}|)$$

A claim is classified as **supported** if $\max_i \text{ClaimScore}(s_j, C_i) \geq 0.12$.

10.3 Reliability Scoring Formulation

The continuous reliability score balances claim coverage with evidence strength:

$$\text{Coverage} = (N_{\text{supported_claims}}) / (N_{\text{total_claims}})$$

$$\text{NormalizedSupport} = \min(1.0, (\text{AverageEvidenceScore}) / (0.35))$$

$$\text{ReliabilityScore} = 0.70 \times \text{Coverage} + 0.30 \times \text{NormalizedSupport}$$

- **Grounding Status:** **supported** ($\text{Coverage} \geq 0.80$), **partially_supported** ($0.50 \leq \text{Coverage} < 0.80$), or **unsupported** ($\text{Coverage} < 0.50$).
- If $\text{ReliabilityScore} < 0.40$, the system flags the response as **low_confidence** and provides a direct 1-click human ticketing escalation option.

11. Experimental Setup

11.1 Hardware and Software Environment

All fine-tuning and live neural inference benchmarks were conducted on a single host workstation:

- **GPU:** NVIDIA GeForce RTX 2050 Laptop GPU (1 device)
- **Total Dedicated VRAM:** 4.00 GB (4096 MB)
- **CUDA Driver / Runtime:** CUDA 12.4
- **Deep Learning Framework:** PyTorch 2.6.0+cu124
- **Model Libraries:** HuggingFace Transformers 5.17.0 [15], PEFT 0.21.0 [16], bitsandbytes 0.50.2
- **Operating System & Runtime:** Windows 11, Python 3.12.10 AMD64

11.2 Evaluation Protocol

Evaluation was performed across the 10 test cases of Frozen Holdout Dataset 2 ($N=10$):

- 7 Answerable Domain Queries ($N_{ans}=7$, Cases 11–17).
- 3 Unsupported / Adversarial Queries ($N_{unsupp}=3$, Cases 18–20).
- Greedily decoded generation (`do_sample=False`, `max_new_tokens=96`).

12. Evaluation Metrics

Metric calculations adhere strictly to the definitions established in `research/EVALUATION_PROTOCOL.md`:

1. Recall@5:

$$\text{Recall@5} = (1) / (N_{ans}) \sum_{i=1}^{N_{ans}} I(\text{c}_i^* \in \text{Top5}(Q_i))$$

2. Mean Reciprocal Rank (MRR):

$$\text{MRR} = (1) / (N_{ans}) \sum_{i=1}^{N_{ans}} (1) / (\text{Rank})(\text{c}_i^*)$$

3. Composite Accuracy:

$$\text{Accuracy} = (N_{correct_answers} + N_{correct_safe_refusals}) / (N_{total})$$

4. Faithfulness:

$$\text{Faithfulness} = (1) / (N_{ans}) \sum_{i=1}^{N_{ans}} \leqft(\sum_{j=1}^{M_i} I(\text{Score}(\text{claim}_{i,j}, C_i) \geq 0.12)) / (M_i) \right)$$

5. Citation Correctness:

$$\text{CitationCorrectness} = (N_{valid_citations_ans} + N_{zero_citations_unsupp}) / (N_{total})$$

6. Hallucination Rate:

$$\text{HallucinationRate} = (N_{hallucinated_unsupported}) / (N_{unsupp})$$

7. Latency Separation:

- **Retrieval & Verification Latency (CPU):** Measured in CPU wall-clock seconds for candidate search and RRF.
- **LLM Generation Latency (GPU):** Measured via synchronized CUDA timers (`torch.cuda.synchronize()`) for token generation.

13. Results

Table 1 presents the empirical evaluation results across model architectures on Frozen Holdout Dataset 2 ($N=10$).

Table 1. Empirical benchmark results on Frozen Holdout Dataset 2 ($N=10$).

Model Architecture	Quantization	Accuracy (N=10)	Faithfulness (N _{ans} =7)	Recall@5 (N _{ans} =7)	MRR (N _{ans} =7)	Citation Corr. (N=10)	Hallucination (N _{unsupp} =3)	LLM Gen. Latency	Peak Inference VRAM
Base Qwen 2.5 0.5B	FP16 (Pretrained)	100.0% (10/10)	95.2%	1.0000	1.0000	100.0% (10/10)	0.0% (0/3)	2.061 s	0.96 GB
SupportIQ LoRA	FP16 PEFT ($r=8$)	100.0% (10/10)	100.0%	1.0000	1.0000	100.0% (10/10)	0.0% (0/3)	0.844 s	0.96 GB

Model Architecture	Quantization	Accuracy (N=10)	Faithfulness (N_ans=7)	Recall@5 (N_ans=7)	MRR (N_ans=7)	Citation Corr. (N=10)	Hallucination (N_unsupp=3)	LLM Gen. Latency	Peak Inference VRAM
SupportIQ QLoRA	4-bit NF4 PEFT	100.0% (10/10)	100.0%	1.0000	1.0000	100.0% (10/10)	0.0% (0/3)	1.668 s	0.46 GB

Table 2. Offline extractive pipeline baseline (Experiment 3, Run 42).

Variant	Evaluation Mode	Accuracy	Faithfulness	Recall@5	MRR	Retr. & Verif. Latency (CPU)	LLM Gen. Latency
RAG + QLoRA Config	Offline Extractive	100.0% (10/10)	98.0%	1.0000	1.0000	0.0119 s	Not measured

As documented in Table 1:

- All three neural configurations achieve 100.0% composite accuracy on the 10 holdout cases by answering all 7 answerable questions correctly and safely refusing all 3 unsupported queries.
- Pretrained Base Qwen achieves 95.2% claim faithfulness due to conversational verbosity, whereas both fine-tuned LoRA and QLoRA achieve 100.0% claim faithfulness, adhering strictly to the extracted facts.
- In the offline extractive pipeline (Table 2), retrieval and verification execute in just 11.9 milliseconds (0.0119 s) on CPU.

14. Ablation Analysis

To quantify the contribution of each algorithmic stage, Experiment 4 isolated individual components across Holdout Dataset 2 ($N=10$). Table 3 details the empirical ablation results.

Table 3. Retrieval and verification component ablation study (Experiment 4, Dataset 2).

Run ID	Ablation Mode	Component Isolated	Accuracy (N=10)	Recall@5 (N_ans=7)	MRR (N_ans=7)	Faithfulness (N_ans=7)	Citation Corr. (N=10)	Hallucination (N_unsupp=3)	Retr. Latency	Key Failure Mechanism
45	BM25 Only	Sparse lexical matching only	90.0% (9/10)	1.0000	1.0000	95.6%	90.0% (9/10)	33.3% (1/3)	0.0052 s	Keyword trap: Case #20 matched "warranty" in Dell chunk
46	Dense Vector Only	32-dim hash vector cosine only	70.0% (7/10)	1.0000	0.7976	76.5%	60.0% (6/10)	100.0% (3/3)	0.0051 s	Semantic drift: Case 16 @ rank 3, Case 17 @ rank 4; 100% hallucination
47	Hybrid w/o RRF	Linear score sum (0.65L + 0.35V)	90.0% (9/10)	1.0000	1.0000	90.8%	90.0% (9/10)	33.3% (1/3)	0.0051 s	Keyword spike: Case #20 lexical score exceeded threshold
48	Hybrid + RRF	Naive RAG (RRF without gate)	70.0% (7/10)	1.0000	1.0000	0.0%	70.0% (7/10)	100.0% (3/3)	0.0051 s	Unconditional generation: 100% hallucination on unsupported
49	Full Retr. + Verif.	RRF + Verif. (Generic filter)	90.0% (9/10)	1.0000	1.0000	98.0%	90.0% (9/10)	33.3% (1/3)	0.0051 s	Boilerplate trap: Case #18 matched terms "supportiq", "support"
50	Full SupportIQ	Hybrid RRF + Dual Safeguard Gate	100.0% (10/10)	1.0000	1.0000	98.0%	100.0% (10/10)	0.0% (0/3)	0.0049 s	None: 100% safe refusal, 0 false positives

Per-Case Error Analysis on Adversarial Queries

- **Case #18 ("Does SupportIQ offer holographic telepathic customer support?"):** Fails in Dense Vector (Run 46), Naive RRF (Run 48), and Generic Verification (Run 49). In Run 49, the query matched common corporate tokens ("supportiq", "customer", "support"). Introducing domain stopword filtering in the Full Pipeline (Run 50) successfully suppressed the spurious match, triggering safe refusal.
- **Case #19 ("Can I pay for my enterprise subscription with Martian mineral mining credits?"):** Fails in Dense Vector Only and Naive RRF due to non-zero distributed hash collisions, but is correctly rejected once lexical grounding is enforced.
- **Case #20 ("What is the warranty coverage for warp drive antimatter core containment breaches?"):** Contains the keyword "warranty", causing BM25 Only (Run 45) and Linear Hybrid (Run 47) to match Dell hardware warranty Chunk #14. The dual safeguard of RRF rank dampening and substantive token filtering successfully eliminated this keyword trap.

15. Discussion

15.1 Operational Trade-Offs: LoRA vs. QLoRA

The empirical evaluations provide concrete insights into the architectural trade-offs between FP16 LoRA and 4-bit NF4 QLoRA under constrained local execution:

- 1. Memory Efficiency:** 4-bit NF4 QLoRA allocated only **0.46 GB of VRAM** during live inference, achieving a **52.1% memory reduction** relative to FP16 LoRA (0.96 GB) and the Base Qwen model (0.96 GB). On the 4.0 GB physical VRAM boundary of the RTX 2050 laptop GPU, QLoRA consumes only 11.5% of total capacity, providing substantial headroom for concurrent system processes or multi-tenant serving.
- 2. Generation Latency:** FP16 LoRA achieved an autoregressive generation latency of **0.844 seconds**, operating **1.98× faster** than QLoRA (1.668 seconds) and **2.44× faster** than the unadapted base model (2.061 seconds). The latency overhead observed in QLoRA stems from the on-the-fly dequantization of 4-bit NF4 weight matrices into FP16 precision during tensor contraction operations.
- 3. Task Performance and Adaptation Fidelity:** Within the evaluated 10-case holdout dataset, both LoRA and QLoRA achieved identical composite accuracy (100.0%) and claim faithfulness (100.0%). These findings indicate that 4-bit NF4 quantization preserved domain task performance without observable degradation relative to FP16 adaptation in this specific benchmark suite.

Crucially, neither configuration represents a universally superior solution: FP16 LoRA is optimal when minimizing user-perceived interactive response time is the primary engineering objective, whereas 4-bit QLoRA is essential when multiple models or background microservices must share a constrained GPU memory envelope.

15.2 The Necessity of Deterministic Grounding Safeguards

The component ablation study demonstrates that neither hybrid retrieval nor non-linear rank fusion alone is sufficient to eliminate generative hallucinations. In Run 48 (Naive RAG), ranking performance on answerable queries was optimal (MRR 1.0000, Recall@5 1.0000); however, the system hallucinated on 100% of unsupported queries because top-ranked candidate chunks were unconditionally passed to generation regardless of relevance. Unaugmented generative models lack intrinsic boundary awareness for out-of-domain queries. Coupling hybrid retrieval with algorithmic evidence gates, sentence-level claim decomposition, and domain stopword filtering is therefore essential for reliable customer support automation.

16. Limitations

To uphold strict scientific integrity, several explicit limitations of this study must be disclosed:

- 1. Holdout Benchmark Scale:** The primary evaluation was conducted on a frozen holdout benchmark of $N=10$ structured test cases (7 answerable, 3 unsupported). While carefully balanced to test both retrieval precision and safe refusal, this small sample size reflects a controlled edge verification suite rather than a large-scale statistical benchmark.
- 2. Hardware Boundary:** All training and inference experiments were executed on a single consumer laptop workstation equipped with an NVIDIA GeForce RTX 2050 Laptop GPU (4.0 GB VRAM). Distributed multi-GPU scaling, high-concurrency request queuing, and server-grade hardware performance were not evaluated.
- 3. Retrieval Representation Limits:** The vector retrieval channel utilizes a 32-dimensional deterministic hash-vector representation rather than a learned continuous neural embedding model (e.g., Sentence-Transformers or DPR). While computationally lightweight and memory-efficient on CPU, hash vectors cannot capture deep latent semantic synonyms.
- 4. Domain Scope:** The experimental knowledge base comprises seven corporate policy documents (14 discrete chunks). The operational dynamics of this pipeline across vast, multi-million-page documentation lakes or dynamically streaming knowledge graphs remain unmeasured.
- 5. Evaluation Scope:** The current benchmark focuses on single-turn question-answering interactions and does not evaluate multi-turn conversational state tracking or complex slot-filling dialogue.
- 6. Generalization Constraints:** The empirical results observed in this work should not be generalized beyond the evaluated hardware constraints, dataset scale, and policy domain without further comprehensive testing.

17. Conclusion

This paper presented SupportIQ, an end-to-end question-answering framework for customer support that couples two-stage hybrid retrieval with parameter-efficient fine-tuning and deterministic claim-level verification. By combining sparse BM25 token matching with a 32-dimensional deterministic hash-vector representation via Reciprocal Rank Fusion ($k=60$), SupportIQ reliably retrieves policy evidence while suppressing keyword traps. Fine-tuning Qwen/Qwen2.5-0.5B-Instruct with LoRA and QLoRA updated only 0.1093% of parameters while achieving 100.0% composite accuracy and 100.0% claim faithfulness on the evaluated frozen holdout benchmark.

Our empirical findings demonstrate clear operational trade-offs: 4-bit NF4 QLoRA reduces peak inference VRAM to 0.46 GB (a 52.1% saving over unquantized baselines), making it suitable for severely memory-constrained edge hardware, while FP16 LoRA achieves lower generation latency (0.844 s). Crucially, the ablation experiments confirm that hybrid retrieval must be paired with post-generation claim verification, domain stopword filtering, and algorithmic evidence gating to achieve safe abstention on unsupported queries and dependable customer support automation.

18. Future Work

Future extensions of the SupportIQ architecture include:

- 1. Lightweight Learned Neural Bi-Encoders:** Integrating compact, locally quantized neural bi-encoder models (e.g., MiniLM) to replace or augment the 32-dimensional deterministic hash vectors and evaluate semantic retrieval gains under edge memory limits.
- 2. Expanded Benchmark Scale:** Expanding the evaluation suite to hundreds of multi-turn customer dialogues, including noisy and multilingual support interactions.
- 3. Broader Document Domains:** Evaluating pipeline scalability across multi-thousand-page technical documentation libraries and heterogeneous semi-structured formats.
- 4. CRM and Ticketing Integrations:** Connecting the verification engine directly to enterprise ticketing APIs (e.g., Zendesk, Jira) for automated ticket creation, routing, and triage.
- 5. Human-in-the-Loop Feedback Optimization:** Incorporating explicit agent feedback from resolved support tickets into iterative Direct Preference Optimization (DPO) pipelines.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.
- [3] T. Detrmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 10088–10115, 2023.
- [4] G. V. Cormack, C. L. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Res. Dev. Inf. Retr.*, pp. 758–759, 2009.

- [5] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.
- [6] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou, C. Li, C. Li, D. Liu, F. Huang, et al. (Qwen Team), "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.
- [7] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, "FEVER: A large-scale dataset for fact extraction and VERification," in *Proc. Conf. N. Am. Chapter Assoc. Comput. Linguist. Hum. Lang. Technol. (NAACL-HLT)*, pp. 809–819, 2018.
- [8] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
- [9] Y. Gao, Y. Xiong, X. Wang, K. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.
- [10] K. Shuster, S. Poff, M. Chen, D. Kiela, and J. Weston, "Retrieval augmentation reduces hallucination in conversation," in *Findings of the Association for Computational Linguistics (EMNLP)*, pp. 3784–3803, 2021.
- [11] V. Karpukhin, B. Ouz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP)*, pp. 6769–6781, 2020.
- [12] A. Gautam, M. Venkatesh, and B. Mascarenhas, "Domain-specific conversational AI in customer support: Challenges and opportunities," *IEEE Access*, vol. 10, pp. 45120–45135, 2022.
- [13] O. Yoran, T. Wolfson, O. Ram, and J. Berant, "Making retrieval-augmented language models robust to irrelevant context," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
- [14] B. Bohnet, V. Q. Tran, P. Kandpal, J. Alberti, C. Sun, and M. Collins, "Attributed question answering: Evaluation and modeling for attributed large language models," *arXiv preprint arXiv:2212.08037*, 2022.
- [15] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, et al., "Transformers: State-of-the-art natural language processing," in *Proc. Conf. Empirical Methods Nat. Lang. Process. Syst. Demonstrations*, pp. 38–45, 2020.
- [16] S. Mangrulkar, S. Gugger, L. Debut, Y. von Platen, T. Wolf, and M. Bossan, "PEFT: State-of-the-art parameter-efficient fine-tuning methods," *Hugging Face*, 2022. [Online]. Available: <https://github.com/huggingface/peft>